

An Algebraic Specification of `fix-hostfiles`

September 5, 2026

Abstract

This document specifies the externally significant behavior of the C implementation of `fix-hostfiles`. It defines the program state, primitive transformations, command semantics, algebraic laws, failure boundaries, and the refinement map to the C modules. Particular attention is given to exact DNS matching, observational idempotence, backup history, and atomic replacement of the active hosts file.

Contents

1	Purpose and Scope	2
1.1	Notation	2
2	Sorts and Abstract State	3
2.1	Sorts	3
2.2	State	3
2.3	Execution	4
3	Equality and Primitive Transformations	4
3.1	Equality Relations	4
3.2	Tokens Before a Comment	5
3.3	Hosts-File Removal	5
3.4	Allow-List Insertion	5
4	Core Laws	5
5	Commands and Action Semantics	6
5.1	Well-Formed Commands	6
5.2	Add	7
5.3	Prep	7
5.4	Restore	7
5.5	Flush	8
6	Action Laws and Their Limits	8
7	Failure and Atomicity	9
7.1	Atomic Visibility of Hosts Replacement	9
7.2	Atomic Visibility Is Not Full Crash Durability	10
7.3	Add Is Not a Multi-File Transaction	10

7.4	Prep and Restore Copy in Place	10
8	Refinement Map to the C Implementation	10
8.1	Command Refinement	11
8.2	Exact-Match Refinement	11
8.3	Subprocess Refinement	11
8.4	Excluded Implementation Details	12
9	Testing Consequences and Design Questions	12
9.1	Properties Suggested for Testing	12
9.2	Round-Trip and Noninterference Properties	13
9.3	Failure Injection	13
9.4	Design Questions Exposed by the Algebra	13
10	Summary	14

1 Purpose and Scope

This specification describes the C implementation of `fix-hostfiles` as an algebra of state transformations. It is not a transcription of the source code. Implementation details appear only where they refine, constrain, or qualify the abstract behavior.

The program coordinates four operations around `hblock(1)` and the macOS hosts file:

prep Preserve the active hosts file and run `hblock`.

restore Replace the active hosts file with the preserved original.

add Add a DNS name to the allow list and remove its active mappings.

flush Flush the DNS cache and restart `mDNSResponder`.

The algebra has four purposes:

1. define the state visible at the program boundary;
2. state precise laws for exact matching, idempotence, commutativity, and reversibility;
3. distinguish the laws of primitive transformations from the weaker guarantees of multi-file commands; and
4. provide properties that can be checked independently of individual implementation examples.

The specification deliberately separates several kinds of equality. An operation can be idempotent with respect to the active hosts file and allow list while still changing backup history. Similarly, an atomic rename can protect one pathname without making the complete command transactional.

1.1 Notation

Function application is written as $f(x)$. Composition is written as $f \circ g$, so $(f \circ g)(x) = f(g(x))$. A finite sequence is written using angle brackets, and $++$ denotes concatenation. State field replacement

uses the notation

$$S[H := H', A := A'].$$

Every field not named in a replacement retains its old value. Equations are interpreted over successful executions unless a failure result is shown explicitly.

2 Sorts and Abstract State

2.1 Sorts

The signature contains the following principal sorts:

$$\begin{aligned} \text{Bool} &= \{\text{false}, \text{true}\}, \\ \text{Result} &= \{\text{success}, \text{failure}, \text{cancelled}\}, \\ \text{Action} &= \{\text{prep}, \text{restore}, \text{add}, \text{flush}\}, \\ \text{Verbosity} &= \{\text{quiet}, \text{verbose}\}. \end{aligned}$$

The remaining sorts are characterized rather than exhaustively enumerated:

DNSName A syntactically valid fully qualified DNS name.

Line A finite sequence of characters.

Hosts A finite sequence of hosts-file lines.

AllowList A finite sequence of DNS-name lines.

Backup Either `none` or `some( $H$ )` for some $H \in \text{Hosts}$.

Metadata The mode, owner ID, and group ID of the active hosts file.

Trace A finite sequence of diagnostic messages.

Cache An abstract state of the DNS cache and `mDNSResponder`.

Hosts and **AllowList** are sequences rather than sets because their on-disk ordering is observable. A separate set interpretation is used when only membership matters.

2.2 State

The abstract state is the product

$$\text{State} = \text{Hosts} \times \text{Backup} \times \text{AllowList} \times \text{Backup} \times \text{Metadata} \times \text{Cache}.$$

A state is written

$$S = (H, O, A, B, M, K),$$

where:

H	active hosts-file contents
O	contents of <code>hosts-ORIG</code> , or <code>none</code> when absent
A	hblock allow-list contents
B	contents of <code>hosts.bak</code> , or <code>none</code> when absent
M	mode and ownership of the active hosts file
K	DNS-cache and responder state

Field selection is written $S.H$, $S.O$, and so forth. Real execution also contains paths, directory entries, timestamps, process identifiers, file descriptors, and possible faults. Those details enter the refinement and failure models rather than every state equation.

2.3 Execution

A parsed command produces a result, a new state, and an output trace:

$$\text{Exec} : \text{Command} \times \text{State} \longrightarrow \text{Result} \times \text{State} \times \text{Trace}.$$

The trace includes ordinary progress output, verbose metadata, and error diagnostics. Separating the trace from the state permits a precise statement that verbose execution should not change the filesystem result.

3 Equality and Primitive Transformations

3.1 Equality Relations

Definition 3.1 (Full equality). $S = T$ requires equality of every modeled field, including backup history and allow-list ordering.

Definition 3.2 (Hosts-and-allow-list equivalence).

$$S \approx_{HA} T \iff S.H = T.H \wedge S.A = T.A.$$

This relation ignores backups, metadata, cache state, and traces.

Let $\text{set}(A)$ discard ordering and duplicate lines from an allow list.

Definition 3.3 (Setwise equivalence).

$$S \approx_{\text{set}} T \iff S.H = T.H \wedge \text{set}(S.A) = \text{set}(T.A).$$

These relations prevent the word *idempotent* from concealing which parts of an execution are actually being observed.

3.2 Tokens Before a Comment

`tokens(l)` splits a line on whitespace and stops at the first hash character. For example,

```
tokens("0.0.0.0 ads.example.com tracker.example.com")
  = ⟨"0.0.0.0", "ads.example.com", "tracker.example.com"⟩,

tokens("0.0.0.0 ads.example.com # local")
  = ⟨"0.0.0.0", "ads.example.com"⟩,

tokens("# ads.example.com") = ⟨⟩.
```

Exact active membership is defined by

$$\text{contains}(d, l) \iff d \in \text{tokens}(l).$$

Thus `ads.example.com` is not contained in a line whose only relevant token is `notads.example.com`.

3.3 Hosts-File Removal

For $d \in \text{DNSName}$, $R_d : \text{Hosts} \rightarrow \text{Hosts}$ removes every line containing d as an exact active token:

$$R_d(\langle \rangle) = \langle \rangle,$$

$$R_d(l :: hs) = \begin{cases} R_d(hs), & \text{contains}(d, l), \\ l :: R_d(hs), & \neg \text{contains}(d, l). \end{cases}$$

Retained lines preserve both relative order and byte contents.

3.4 Allow-List Insertion

For $d \in \text{DNSName}$, $I_d : \text{AllowList} \rightarrow \text{AllowList}$ appends d only when an exactly equal line is absent:

$$I_d(A) = \begin{cases} A, & d \in \text{lines}(A), \\ A ++ \langle d \rangle, & d \notin \text{lines}(A). \end{cases}$$

The complete-line interpretation corresponds to the implementation's use of `fileContainsLine()` rather than substring search.

4 Core Laws

Law 4.1 (Removal idempotence).

$$R_d(R_d(H)) = R_d(H).$$

After every line containing d has been removed, another removal has no further effect.

Law 4.2 (Commutativity of distinct removals). *For distinct DNS names d and e ,*

$$R_d(R_e(H)) = R_e(R_d(H)).$$

The law remains true when one line contains both names; that line is absent from either final result.

Law 4.3 (Preservation of unrelated lines). *If $\neg \text{contains}(d, l)$, then*

$$\text{count}(l, R_d(H)) = \text{count}(l, H).$$

The law covers blank lines, comments, and tokens that merely contain similar text.

Law 4.4 (Insertion idempotence).

$$I_d(I_d(A)) = I_d(A).$$

Law 4.5 (Setwise commutativity of insertion). *For distinct d and e ,*

$$\text{set}(I_d(I_e(A))) = \text{set}(I_e(I_d(A))).$$

Sequence equality does not generally hold. If both names are initially absent, the two sides append them in opposite orders.

Law 4.6 (Exactness). *For distinct DNS tokens d and e ,*

$$e \in \text{tokens}(l) \wedge d \notin \text{tokens}(l) \implies l \text{ occurs unchanged in } R_d(H).$$

This is the formal reason that removing `ads.example.com` must preserve a line containing `notads.example.com`.

The first four laws admit direct structural proofs from the definitions in the previous section. They also translate naturally into property-based tests.

5 Commands and Action Semantics

5.1 Well-Formed Commands

A parsed command is a triple

$$c = (\text{action}, \text{dns}, \text{verbosity}).$$

The well-formedness predicate requires exactly one action and requires a DNS argument exactly when that action is `add`:

$$\begin{aligned} \text{wellFormed}(c) \iff & c.\text{action} \in \text{Action} \\ & \wedge (c.\text{action} = \text{add} \iff c.\text{dns} = \text{some}(d) \wedge \text{valid}(d)). \end{aligned}$$

Verbosity is a modifier, not an action. Short and long forms are aliases:

$$\begin{aligned}
\text{parse}(\text{"-a"}, d) &= \text{parse}(\text{"-add"}, d), \\
\text{parse}(\text{"-f"}) &= \text{parse}(\text{"-flush"}), \\
\text{parse}(\text{"-h"}) &= \text{parse}(\text{"-help"}), \\
\text{parse}(\text{"-v"}, a) &= \text{parse}(\text{"-verbose"}, a).
\end{aligned}$$

Any command containing two actions is rejected, for example

$$\text{parse}(\text{"-flush prep"}) = \text{failure}.$$

5.2 Add

For a valid DNS name d , the successful abstract effect is

$$\text{Add}_d(S) = S[A := I_d(S.A), B := \text{some}(S.H), H := R_d(S.H)].$$

The active hosts file is backed up immediately before replacement, and its metadata is preserved:

$$\text{Add}_d(S).M = S.M.$$

An invalid name has no intended state transition:

$$\neg \text{valid}(d) \implies \text{Exec}(\text{add}(d), S) = (\text{failure}, S, t)$$

for some diagnostic trace t .

5.3 Prep

Let $\text{HB}(A, E)$ denote the hosts contents produced by the external `hblock` program from allow list A and external environment E . Subject to successful confirmation when O already exists,

$$\text{Prep}_E(S) = S[O := \text{some}(S.H), H := \text{HB}(S.A, E)].$$

The environment parameter represents block sources, network results, hblock version, and configuration outside this repository.

5.4 Restore

Restore is defined only when the original backup exists:

$$\text{Restore}(S) = \begin{cases} S[H := H_O], & S.O = \text{some}(H_O), \\ \text{failure}, & S.O = \text{none}. \end{cases}$$

Existing active contents require confirmation before replacement.

5.5 Flush

Let `fresh` denote the desired cache state with a running `mDNSResponder`:

$$\text{Flush}(S) = \begin{cases} S[K := \text{fresh}], & \text{on macOS and command success,} \\ \text{failure}, & \text{on other platforms or command failure.} \end{cases}$$

Flush does not intentionally modify H , O , A , B , or M .

6 Action Laws and Their Limits

Law 6.1 (Observational idempotence of add). *For valid d ,*

$$\text{Add}_d(\text{Add}_d(S)) \approx_{HA} \text{Add}_d(S).$$

The allow list contains one copy of d , and the active hosts file contains no line with an active d token.

Boundary 6.2 (Add is not fully idempotent). Under full state equality,

$$\text{Add}_d(\text{Add}_d(S)) \neq \text{Add}_d(S)$$

in general. After the first call, $B = \text{some}(S.H)$. After the second call, $B = \text{some}(R_d(S.H))$.

The behavior test for repeated addition proves idempotence of the allow list and active hosts result. It does not require preservation of the first backup. The distinction is semantic idempotence versus backup-history idempotence.

Law 6.3 (Setwise commutativity of distinct adds). *For valid, distinct d and e ,*

$$\text{Add}_d(\text{Add}_e(S)) \approx_{set} \text{Add}_e(\text{Add}_d(S)).$$

Full equality can fail because appended allow-list order differs and because `hosts.bak` records the hosts file immediately before the final add.

Law 6.4 (Restore idempotence). *If the original backup remains present and unchanged, and both confirmations succeed, then*

$$\text{Restore}(\text{Restore}(S)) = \text{Restore}(S).$$

The model omits timestamps that can differ between physical copies.

Law 6.5 (Prep-restore round trip). *If prep succeeds and O is not modified before restore, then*

$$\text{Restore}(\text{Prep}_E(S)).H = S.H.$$

This is a round-trip law for active contents, not necessarily for the complete state. Cache state and physical timestamps can differ.

Boundary 6.6 (Prep is not idempotent). In general,

$$\text{Prep}_E(\text{Prep}_E(S)) \neq \text{Prep}_E(S).$$

A confirmed second preparation overwrites `hosts-ORIG` with the current hblock-generated hosts file. That changes the restoration point. The confirmation prompt is therefore semantically important rather than merely a user-interface convenience.

Law 6.7 (Verbose noninterference). *Let $q(c)$ and $v(c)$ differ only in verbosity. For any successful command,*

$$\text{state}(\text{Exec}(q(c), S)) = \text{state}(\text{Exec}(v(c), S)).$$

Only the trace is expected to differ:

$$\text{trace}(\text{Exec}(q(c), S)) \neq \text{trace}(\text{Exec}(v(c), S)).$$

The model ignores incidental access-time changes a filesystem might make while objects are inspected.

Law 6.8 (Abstract idempotence of flush).

$$\text{Flush}(\text{Flush}(S)).K = \text{fresh}.$$

Process identifiers, timing, and diagnostics may differ, so the law applies only to the abstract cache state.

7 Failure and Atomicity

7.1 Atomic Visibility of Hosts Replacement

During the final replacement performed by `removeHostEntry()`, an observer of the active hosts pathname sees one of two complete values:

$$\text{visible}(H) \in \{H_{old}, R_d(H_{old})\}.$$

No observer should see a partially generated $R_d(H_{old})$. The implementation creates the temporary file in the same directory, writes the complete retained contents, calls `fflush()` and `fsync()`, restores metadata, closes the stream, and then calls `rename()`.

Listing 1: Core atomic replacement sequence

```
temporary_fd = mkstemp(temporary_path);
destination = fdopen(temporary_fd, "w");

/* Write the complete revised file. */

if (fflush(destination) == EOF ||
    fsync(fileno(destination)) == -1 ||
    fchmod(fileno(destination),
           hosts_status.st_mode & 07777) == -1)
    goto cleanup;

if (rename(temporary_path, hosts_path) == -1)
    goto cleanup;
```

If any step before `rename()` fails, the active hosts contents remain H_{old} and `cleanup` unlinks the incomplete temporary file.

7.2 Atomic Visibility Is Not Full Crash Durability

The replacement file is synchronized before rename. The containing directory is not synchronized after rename. Consequently, the implementation provides atomic pathname visibility but does not claim the strongest possible guarantee that the new directory entry survives immediate power loss.

This distinction can be stated as:

$$\text{atomicVisibility} \not\Rightarrow \text{maximumCrashDurability}.$$

A directory `fsync()` after rename would strengthen the durability contract.

7.3 Add Is Not a Multi-File Transaction

The allow list changes before the hosts file is replaced. A later failure can therefore leave

$$d \in \text{set}(A) \quad \wedge \quad H = H_{old}.$$

Likewise, `hosts.bak` can be updated even if temporary-file generation or rename later fails. Atomic replacement protects the active hosts pathname; it does not make the complete `add` action all-or-nothing.

7.4 Prep and Restore Copy in Place

`copyFile()` opens its destination with truncation. The `prep` and `restore` copies are therefore outside the atomic-replacement law. A failure during either copy can leave a partially written destination.

Observation 7.1. Separating primitive laws from whole-command behavior reveals three different levels of assurance:

1. exact and idempotent content transformation;
2. atomic replacement of one pathname; and
3. transactional update of several files.

The current implementation provides the first two for hosts-file removal, but does not claim the third.

8 Refinement Map to the C Implementation

The abstract operators correspond to C types and functions as follows:

Algebra	C implementation
Command	Command in <code>src/actions.h</code>
State paths	Paths in <code>src/fix-hosts.h</code>
<code>contains(d, l)</code>	<code>lineContainsHost()</code>
<code>R_d(H)</code>	filtering loop in <code>removeHostEntry()</code>

Algebra	C implementation
$I_d(A)$	<code>fileContainsLine()</code> and <code>appendLine()</code>
Add_d	<code>addDnsName()</code>
Prep, Restore	<code>updateHostsFiles()</code>
Flush	<code>flushDnsCache()</code>
atomic replacement	<code>mkstemp()</code> , <code>fsync()</code> , and <code>rename()</code>
metadata preservation	<code>fchmod()</code> and <code>fchown()</code>
trace extension	<code>fileInfo()</code> and verbose reporting
external command execution	<code>runCommand()</code>

8.1 Command Refinement

The C command state is:

Listing 2: Parsed command state

```
typedef struct {
    Action action;
    const char *dns_name;
    bool verbose;
} Command;
```

`selectAction()` enforces the one-action invariant. Positional `prep` and `restore` requests pass through the same check as `-add` and `-flush`. The `verbose` field refines the trace component rather than the filesystem state.

8.2 Exact-Match Refinement

`lineContainsHost()` scans whitespace-separated tokens and stops at a comment. Equality requires both equal length and `strncmp()` equality:

Listing 3: Exact DNS-token comparison

```
if (token_length == expected_length &&
    strncmp((const char *)token,
            dns_name, expected_length) == 0)
    return true;
```

The length check is essential. Without it, a prefix could satisfy the byte comparison and violate Law 4.6.

8.3 Subprocess Refinement

`runCommand()` implements external operations with `fork()`, `execvp()`, and `waitpid()` rather than a shell command string.

Listing 4: Argument-vector execution

```
char *const flush_arguments[] = {
    "dscacheutil", "-flushcache", NULL
};

if (runCommand(flush_arguments) != 0)
    return EXIT_FAILURE;
```

This refinement passes arguments directly to the executable, eliminating an otherwise unnecessary shell grammar from the command semantics.

8.4 Excluded Implementation Details

The refinement map intentionally omits exact message wording, output spacing, timestamps, allocation strategy, and Doxygen text. Those details can be specified separately, but they do not alter the principal state transitions defined here.

9 Testing Consequences and Design Questions

9.1 Properties Suggested for Testing

The existing behavior tests exercise representative examples. The algebra suggests a complementary property-based suite.

For generated hosts files and valid names d and e , check:

$$\begin{aligned} R_d(R_d(H)) &= R_d(H), \\ R_d(R_e(H)) &= R_e(R_d(H)), \\ \text{Add}_d(\text{Add}_d(S)) &\approx_{HA} \text{Add}_d(S). \end{aligned}$$

Generated hosts lines should include:

- prefix and suffix lookalikes;
- multiple hostnames on one line;
- tabs and repeated spaces;
- comments containing the target name;
- a target immediately followed by a hash character;
- blank lines; and
- files without a final newline.

For allow-list insertion, check:

$$\text{count}(d, \text{Add}_d(S).A) = 1.$$

A separate test should document the current second-add backup behavior rather than accidentally treating full-state idempotence as guaranteed.

9.2 Round-Trip and Noninterference Properties

With a deterministic `hblock` stub,

$$\text{Restore}(\text{Prep}(S)).H = S.H.$$

Repeated prep should also be tested explicitly because it changes the restoration point after confirmation.

Run equivalent fixtures in quiet and verbose modes, then compare all modeled files and metadata:

$$\text{state}(\text{Exec}(q(c), S)) = \text{state}(\text{Exec}(v(c), S)).$$

9.3 Failure Injection

Inject failures into append, copy, write, synchronize, permission, ownership, close, and rename steps. Verify the exact boundary promised by each operation, especially

$$\text{failureBeforeRename} \implies H = H_{old}.$$

Failure tests should also record permitted partial progress in the allow list and backup files.

9.4 Design Questions Exposed by the Algebra

The specification raises several concrete choices:

1. Should repeated `Addd` preserve the first `hosts.bak`, or should the backup always represent the state immediately before the latest invocation?
2. Should allow-list insertion preserve append order, or should the file be treated as a mathematical set and normalized?
3. Should prep refuse to overwrite `hosts-ORIG` rather than permit an interactive overwrite?
4. Should `copyFile()` use the same atomic replacement primitive as hosts-file removal?
5. Should add become transactional across the allow list, backup, and active hosts file, or is recoverable partial progress acceptable?
6. Should the containing directory be synchronized after rename for stronger crash durability?

None of these questions proves the implementation incorrect. Each identifies a semantic decision that can be stated, tested, and documented precisely.

10 Summary

The central algebra is compact:

$$\begin{aligned}
 I_d(A) &= \begin{cases} A, & d \in \text{lines}(A), \\ A ++ \langle d \rangle, & d \notin \text{lines}(A), \end{cases} \\
 R_d(H) &= \text{filter}(\text{lines not containing } d \text{ as an exact token}, H), \\
 \text{Add}_d(S) &= S[A := I_d(S.A), B := \text{some}(S.H), H := R_d(S.H)], \\
 \text{Prep}_E(S) &= S[O := \text{some}(S.H), H := \text{HB}(S.A, E)], \\
 \text{Restore}(S) &= S[H := H_O] \quad \text{when } S.O = \text{some}(H_O), \\
 \text{Flush}(S) &= S[K := \text{fresh}].
 \end{aligned}$$

Its principal laws are:

$$\begin{aligned}
 R_d \circ R_d &= R_d, \\
 R_d \circ R_e &= R_e \circ R_d, \\
 I_d \circ I_d &= I_d, \\
 \text{Add}_d(\text{Add}_d(S)) &\approx_{HA} \text{Add}_d(S), \\
 \text{Restore}(\text{Prep}_E(S)).H &= S.H.
 \end{aligned}$$

Quiet and verbose forms produce the same modeled state while producing different traces.

The most important qualifications are:

- Add is not fully idempotent when `hosts.bak` is observed.
- Distinct adds commute setwise, but not in file order or backup history.
- Prep is not idempotent because it advances the restoration point.
- Atomic rename protects one pathname, not the complete multi-file operation.
- Atomic visibility does not by itself imply maximum crash durability.

These qualifications are where the algebra adds the most value. Familiar operational claims become precise enough to verify, refute, or deliberately adopt as part of the program's contract.